COMPUTER ENGINEERING
QUEZON CITY

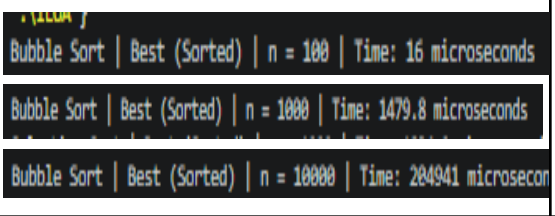
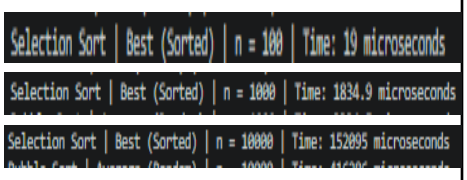
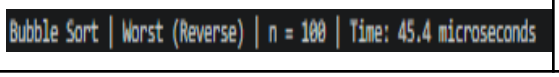
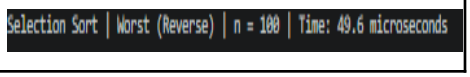
| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 - Data Structures and Algorithm | **SECTION:** | CPE21S1 |
| **DATE PERFORMED:** | 10-20-2026 | **DATE SUBMITTED:** | 10-22-2026 |
| **NAME:** | Canoy Hail B.<br>Mauro, Ryan Lewis B. | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

## ACTIVITY NO. 14.1 Algorithm Complexity

### 1. PROCEDURE OUTPUT

| | Best Case | Worst Case | Analysis |
|---|---|---|---|
| **Algorithm 1: Bubble Sort** | **O(n) - when the values are already sorted the algorithm uses a flag to stop early because no swaps are no longer needed.** | **O(n^2) - When the list of values is reversed it may require a lot of swaps.** | **Bubble Sort is efficient in a small and almost sorted list of data but its efficiency decreases as the list of values increases along with the unsorted values.** |
| **Algorithm 2: Selection Sort** | **O(n^2) - It still searches the remaining element to find the minimum even though the list is already sorted.** | **O(n^2) - it must scan the remaining element each time to find the minimum.** | **Selection Sort has quadratic time complexity because the number depends on its input size rather than its initial order. It checks all the remaining elements to find the smallest numbers in every pass.** |

**Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools**

| | Algorithm 1 | Algorithm 2 |
|---|---|---|
| **Best Case** |  |  |
| **Worst Case** |  |  |

| | | |
|---|---|---|
| | Bubble Sort \| Worst (Reverse) \| n = 1000 \| Time: 3149.3 microseconds<br>Bubble Sort \| Worst (Reverse) \| n = 10000 \| Time: 280263 microseconds | Selection Sort \| Worst (Reverse) \| n = 1000 \| Time: 1885.3 microsecond<br>Selection Sort \| Worst (Reverse) \| n = 10000 \| Time: 129325 microseconds |

**Table 14-2. Best- and Worst-Case Analysis using Experimental Tools**

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>
#include <chrono>

// Linear Search implementation from searchingAlgo.h
template <typename T>
void linearSearchArr(T data[], int arrSize, T dataFind){
    int i = 0;
    while(i < arrSize){
        if(dataFind == data[i]){
            return;
        }
        i++;
    }
}

// Helper function for Binary Search
template <typename T>
int getMiddle(T data[], int first, int last){
    return first + (last - first) / 2;
}

// Binary Search implementation from searchingAlgo.h
template <typename T>
void binarySearchArr(T data[], int arrSize, T findData){
    int first = 0;
    int last = arrSize - 1;

    while(first <= last){
        int middle = getMiddle(data, first, last);

        if(data[middle] == findData){
            return;
        }
        if(data[middle] < findData){
            first = middle + 1;
        } else {
            last = middle - 1;
        }
    }
}
```

```cpp
// Benchmark function for worst-case scenario
void benchmarkSearch(int n) {
    int* arr = new int[n];
    for(int i = 0; i < n; i++) {
        arr[i] = i; // Populating sorted dataset
    }

    // Worst-case target: A value larger than any element in the array
    // forces both searches to traverse/examine the maximum possible steps.
    int target = n + 1;

    // Benchmark Binary Search (Worst-Case)
    auto start = std::chrono::high_resolution_clock::now();
    binarySearchArr(arr, n, target);
    auto end = std::chrono::high_resolution_clock::now();
    double binTime = std::chrono::duration<double, std::milli>(end - start).count();

    // Benchmark Linear Search (Worst-Case)
    start = std::chrono::high_resolution_clock::now();
    linearSearchArr(arr, n, target);
    end = std::chrono::high_resolution_clock::now();
    double linTime = std::chrono::duration<double, std::milli>(end - start).count();

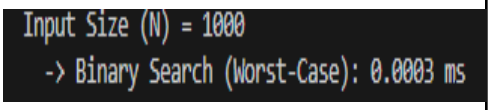
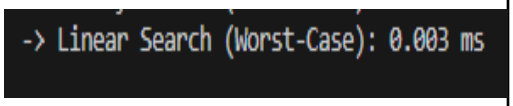
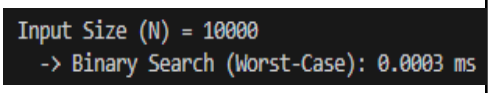
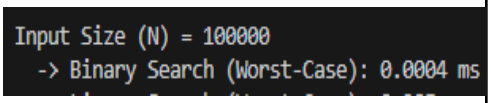
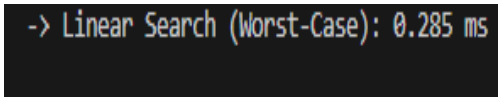
    std::cout << "Input Size (N) = " << n << "\n";
    std::cout << "  -> Binary Search (Worst-Case): " << binTime << " ms\n";
    std::cout << "  -> Linear Search (Worst-Case): " << linTime << " ms\n\n";

    delete[] arr;
}

int main() {
    int sizes[] = {1000, 10000, 100000};

    std::cout << "=== EMPIRICAL SEARCH ANALYSIS (WORST-CASE) ===\n\n";
    for(int n : sizes) {
        benchmarkSearch(n);
    }

    return 0;
}
```

| Input Size (N) | Elapsed Time for Binary Search (Worst-Case) | Elapsed Time for Linear Search (Worst-Case) |
|---|---|---|
| 1,000 |  |  |
| 10,000 |  |  |
| 100,000 |  |  |

| Conclusion (Canoy Hail B.) |
|---|
| In this activity I learned a lot. I learned more about Bubble Sort and Selection Sort how to compare and analyze their speed. I found out that both can take longer when there is more data to sort. Based on the results, Selection Sort was faster in some tests. I also learned that the speed can depend on the computer and the data. This activity helped me understand how to compare the two sorting methods and see which one is faster. |
| **Conclusion (Mauro, Ryan Lewis B)** |
| In this activity, I learned how to compare Bubble Sort and Selection Sort using theoretical and experimental analysis. Both algorithms have O(n^2) time complexity in the code used, which means their execution time generally increases as the input size becomes larger. Based on the experimental results, Selection Sort was faster in some tests, especially with larger input sizes. I also learned that actual execution times may vary depending on the computer and input data. This activity helped me understand the importance of analyzing algorithms to determine their efficiency when sorting different amounts of data. |

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **none**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **none**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

COMPUTER ENGINEERING

QUEZON CITY

**Extent of Use:** **none**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.